

Informe técnico (resumen)

Estado del proyecto Capacity AR al cierre de esta iteración: qué se construyó, qué falta, y riesgos identificados, a la luz de la consigna del modelo pedagógico definida en los documentos del TP.

1. Consigna y alcance

La consigna pide diseñar e implementar un **Sistema de Tutoría Inteligente (ITS)** que capacite a jóvenes de barrios vulnerables de Argentina para insertarse en el sector IT, articulado sobre el marco pedagógico de las **5 P** (Práctica, Paciencia, Perseverancia, Pasión, Play) con esquema **Human-in-the-Loop (HITL)**.

Los documentos de referencia definen el flujo en cinco fases:

- **Fase 0 — Diagnóstico de brecha:** evaluación cruzada entre perfil del estudiante y oferta laboral → `learning path` personalizado.
- **Fase 1 — Pasión:** anclaje motivacional, conectar cada skill con la empresa concreta.
- **Fase 2 — Play:** micro-lecciones, sandbox, exploración sin penalización.
- **Fase 3 — Práctica:** ejercicios adaptativos con umbral de dominio del 80%.
- **Fase 4 — Paciencia y Perseverancia:** refuerzo emocional transversal.

Además, los documentos exigen tres modelos arquitectónicos (Dominio, Estudiante, Pedagógico/multi-agente) y supervisión humana mediante un dashboard con análisis predictivo de riesgo de abandono.

El proyecto se divide en **dos equipos sobre el mismo monolito modular**:

- **Equipo 1 — Gap Engine:** produce el `GapReport` (Fase 0). Recibe los datos crudos del estudiante y de la oferta y genera el reporte de brecha.
- **Equipo 2 — Capacitación + Tutores** (este repo): consume el `GapReport` y produce el plan pedagógico, la ejecución del alumno, el feedback y la supervisión humana (Fases 1 a 4).

Contrato y división de responsabilidades documentados en `scope-equipos.md`.

2. Arquitectura general

Monolito modular en **Python 3.12 + FastAPI + SQLAlchemy 2.x síncrono**, persistido en **PostgreSQL serverless (Neon)** y deployado en **Render** con auto-deploy desde `main`.

Cada módulo en `app/modules/<capability>/` respeta la separación en capas:

Capa	Archivo	Responsabilidad
Router	<code>router.py</code>	Endpoints HTTP, validación de payloads

Capa	Archivo	Responsabilidad
Service	<code>service.py</code>	Reglas de negocio, orquestación, errores
Repository	<code>repository.py</code>	Único lugar con SQLAlchemy
Models	<code>models.py</code>	Modelos declarativos
Schemas	<code>schemas.py</code>	Contratos Pydantic

Los proveedores externos (LLMs, repositorios) se inyectan con el patrón **Protocol + Factory**, lo que permite cambiar de Groq a Gemini (o caer a Mock) sin tocar el resto del código.

Patrón de IA híbrida: el código determinístico arma la estructura del plan (módulos por skill, 3 fases por módulo, 5 ejercicios por fase de Práctica) y el LLM solo rellena título, contenido y opciones de ejercicios. Esto da control sobre la forma del output y permite el fallback automático a Mock si el LLM falla.

Documentación arquitectónica en [backend-onboarding.md](#), [ai-integration.md](#) y [training-module-design.md](#).

3. Lo realizado

3.1 Equipo 1 — Gap Engine

Estado: implementado por Equipo 2 en este repo como `gap_analysis` (porque el equipo 1 no entregó su versión todavía, este módulo lo supe).

Componente	Endpoint	Estado
Upload de 2 documentos (estudiante + oferta)	<code>POST /api/gap-analyses</code> (multipart)	✓
Extracción de texto PDF / DOCX / TXT	<code>gap_analysis/extractors.py</code>	✓
LLM (Groq) extrae <code>GapReport</code> validado con JSON Schema	<code>gap_analysis/service.py</code>	✓
Persistencia del gap pendiente (LIFO por email)	tabla <code>gap_analyses</code>	✓
Disparo del learning path por email	<code>POST /api/students/{email}/generate-learning-path</code>	✓

El flujo está partido en 2 pasos porque la combinación extracción + generación de plan supera el timeout de 30s de Render free tier. El segundo paso busca el último gap pendiente (`learning_path_id IS NULL`) y lo enlaza.

3.2 Equipo 2 — Capacitación

Módulos implementados

Módulo	Endpoints	Cubre
<code>users</code>	CRUD <code>/users</code>	Padrón de actores (student, tutor, company_admin, admin)
<code>skills</code>	CRUD <code>/skills</code> + seed	Catálogo de habilidades IT
<code>user_skills</code>	<code>/students/{email}/skills</code>	Auto-reporte de skills del estudiante
<code>job_descriptions</code>	CRUD <code>/job-descriptions</code>	Ofertas laborales con skills requeridas
<code>gap_analysis</code>	ver arriba	Fase 0 (suplantando Equipo 1)
<code>learning_paths</code>	POST/GET <code>/learning-paths</code>	Fases 1, 2 y 3 del modelo
<code>attempts</code>	POST/GET <code>/attempts</code>	Fase 4 (Paciencia, feedback empático)
<code>resources</code>	(consumido por learning_paths)	Recursos externos por skill+fase

Todos los endpoints están bajo prefijo `/api`.

Cobertura de las 5 P

P	Cómo se implementa	Estado
Pasión	Fase del plan generada por LLM con prompt que conecta la skill con la empresa concreta	✓
Play	Fase del plan con micro-contenido y recursos externos (sandbox, video)	✓
Práctica	Fase del plan con 5 ejercicios multiple_choice A/B/C/D, evaluación con umbral 80%	✓
Paciencia	Feedback empático generado por LLM cuando el alumno falla un ejercicio, sin revelar la respuesta	✓
Perseverancia	Visualización de progreso, streaks, métricas — requiere módulos <code>sessions</code> y <code>student_metrics</code>	✗

Mapeo detallado en [5p-coverage.md](#).

Modelo del Dominio, Estudiante y Pedagógico (según los docx)

- **Modelo del Dominio:** parcialmente implementado vía `skills` + dependencias declaradas en el prompt del LLM. **Falta el grafo formal con prerequisites** (hoy es una lista plana).
- **Modelo del Estudiante:** estado cognitivo está representado por `attempts` y `user_skills`. **Faltan:** rasgos de personalidad (Grit), estado afectivo en tiempo real, contexto socioeconómico estructurado.
- **Modelo Pedagógico (multi-agente):** hoy es un **único agente** (el LLM activo) con prompts especializados por fase. **No hay separación formal** Estratega / Diseñador / Coach.

Integración con IA

Cuatro puntos del sistema usan el LLM activo (Groq por default, Gemini como alternativa, Mock como fallback):

- 1. Extracción del `GapReport` desde texto.
- 2. Generación de units (contenido + 5 ejercicios) por skill.
- 3. Sugerencia de recursos externos por skill y fase, con guardrails anti-alucinación de URLs (whitelist + búsquedas YouTube).
- 4. Feedback empático cuando el alumno falla (Paciencia).

Si el LLM cae, el endpoint sigue funcionando con `generator_used: "mock"`.

Infraestructura y despliegue

- ☒ Producción pública: `https://capacity-ar-ap.onrender.com`.
- ☒ Auto-deploy desde `main`.
- ☒ Documentación de curls de prueba (`api-curls.md`).
- ☒ Guía para frontend (`frontend-guide.md`).

Backlog completado

Resumen de las 6 épicas cerradas:

Épica	Esfuerzo real
C1 — MVP backend FastAPI + módulo users	~4 h
C2 — <code>learning_paths</code> con MockPlanGenerator	~6 h
C3 — Persistencia en PostgreSQL (Neon + Render)	~5 h
C4 — Generación con IA: Groq + Gemini con fallback	~6 h
C5 — <code>attempts</code> con Paciencia (4° P)	~3 h
C6 — <code>resources</code> con anti-alucinación de URLs	~3 h

Total invertido: ~27 horas. Detalle en `backlog.md`.

4. Lo pendiente

4.1 Equipo 1 — Gap Engine "nativo"

Hoy `gap_analysis` está implementado por Equipo 2 como solución pragmática. La consigna original asume que Equipo 1 produce el `GapReport` con un pipeline propio (Google Forms + validación + scoring). Pendiente: definir si esa entrega independiente sucede o si `gap_analysis` queda como solución definitiva.

4.2 Equipo 2 — Backlog priorizado

Las 4 épicas que cierran el modelo 5P + HITL según los documentos del TP:

#	Épica	Por qué importa según la consigna	Esfuerzo
1	Perseverancia y métricas del alumno	Cierra la 5º P. Sin esto, "Perseverancia" queda como concepto sin implementación	3-4 h
2	Tutores y asignaciones	Implementa el HITL exigido por el modelo. Sin esto no hay supervisión humana real	4-5 h
3	Sistema de alertas	El dashboard del tutor con análisis predictivo de abandono es un requisito explícito	3-4 h
4	Next-unit logic y desbloqueo progresivo	Mastery Learning de Bloom: bloquear avance hasta dominar la skill anterior	2-3 h

Total mínimo para cumplir con el modelo de la consigna: **13-16 horas**.

4.3 Otras épicas planificadas

#	Épica	Esfuerzo
5	Validaciones críticas (entregables con ojo humano)	3 h
6	Auth (Fase 6A mock + Fase 6B JWT)	9-12 h
7	Eventos hacia Equipo 1 (contrato append-only)	2 h
8	Calidad técnica (tests, CORS, Alembic)	4-6 h

Detalle, user stories, endpoints y criterios de aceptación de cada épica en [backlog.md](#).

4.4 Brechas conceptuales respecto a la consigna

Punto de la consigna	Brecha actual
Grafo de conocimientos con prerequisites formales	Skills hoy son lista plana; el orden lo decide el priority del GapReport
Modelo Pedagógico multi-agente (Estratega + Diseñador + Coach)	Hoy es un único LLM con prompts por fase
Estado afectivo en tiempo real (detección de frustración vía NLP)	No implementado
Contexto socioeconómico estructurado del alumno	No registrado en BD
Métricas psicométricas (Escala de Grit, SDT)	No implementado
Plan de validación empírica (piloto con grupo control)	No diseñado todavía
Gobernanza de datos y consentimiento informado	No implementado (relevante porque trabajamos con menores en barrios vulnerables)

Punto de la consigna	Brecha actual
Capacitación del tutor humano (onboarding del dashboard)	No diseñada

Estas brechas no son bloqueantes para el TP, pero **deben mencionarse en la presentación** porque están explícitas en los documentos del modelo.

5. Decisiones técnicas relevantes

- **Auth diferida** intencionalmente para no bloquear el resto. Identificación por email mientras tanto. Matriz de permisos ya documentada en [access-matrix.md](#).
- **IA híbrida** (código fija la estructura, LLM solo rellena contenido) en vez de pipeline 100% LLM. Permite controlar la forma del output y caer a Mock sin romper.
- **Identificación compuesta de ejercicios** (`learning_path_id + module_index + unit_index + exercise_index`) en vez de tabla propia. Trade-off documentado en [training-module-design.md](#).
- **Gap analysis partido en 2 pasos** por el timeout de 30s de Render free tier.
- **Anti-alucinación de URLs en recursos**: para videos siempre URLs de búsqueda en YouTube; para docs canónicas, whitelist de dominios.
- **No remover el fallback a Mock**: la API nunca debe romperse por culpa del LLM.

6. Riesgos

Riesgo	Mitigación actual	Mitigación pendiente
Timeout de Render free tier (30s)	Gap analysis partido en 2 pasos	Mover generación de plan a background task con <code>202 Accepted</code>
Cuota de Groq (~14.4k RPD) y Gemini (1.5k RPD)	Fallback a Mock + cache de recursos	Cache de planes por (skill, target_role) si crece el tráfico
Sin tests automatizados todavía	Manual via Swagger + curls	Épica 8 (pytest + httpx)
Sin Alembic (uso de <code>create_all</code>)	Funcional en MVP	Alembic antes del primer cambio de schema con datos reales
Credenciales en variables de entorno de Render	OK	Rotar las claves Groq + Neon antes de la presentación pública
Auth ausente	Endpoints públicos en MVP	Épica 6A (headers mock) desbloquea el resto del backlog
PDFs escaneados (sin OCR)	Rechazo con <code>422</code> (texto muy corto)	Documentado como limitación

7. Próximos pasos recomendados

Para la próxima iteración, ordenados por impacto en la consigna:

1. **Épica 1 (Perseverancia)**: cierra las 5 P. 3-4 h.
 2. **Épica 2 (Tutores)**: activa el HITL exigido por la consigna. 4-5 h.
 3. **Épica 3 (Alertas)**: completa el dashboard predictivo. 3-4 h.
 4. **Épica 4 (Next-unit logic)**: Mastery Learning real con bloqueo de avance. 2-3 h.
 5. **Épica 6A (Auth mock)**: hace efectiva la matriz de accesos documentada. 3-4 h.
 6. **Épica 8 (Tests + CORS + Alembic)**: indispensable para defender el TP como producto, no como prototipo. 4-6 h.
-

8. Referencias

- Modelo pedagógico y arquitectura: [Modelo IA para Capacitación IT.docx](#), [Scrum_Trabajo_Practico.docx](#), [Modelo Capacitacion con IA 2.docx](#).
- Cobertura 5P: [5p-coverage.md](#).
- Contrato entre equipos: [scope-equipos.md](#).
- Arquitectura backend: [backend-onboarding.md](#).
- Integración con IA: [ai-integration.md](#).
- Backlog completo: [backlog.md](#).
- Guía para el frontend: [frontend-guide.md](#).
- Curls de prueba: [api-curls.md](#).
- Matriz de accesos: [access-matrix.md](#).
- Repositorio: <https://github.com/marcelocassiovieira/capacitaya>
- Producción: <https://capacity-ar-ap.onrender.com>